STARTUPHUB AI: AN AI-POWERED STARTUP VALIDATION AND COLLABORATION PLATFORM

A Project Report submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted By:

- **Student Name:** Pranav Krishna
- **Registration Number:** [Registration Number]

Under the Guidance of:

- **Project Guide:** [Project Guide Name / Designation]

[COLLEGE LOGO

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

[UNIVERSITY / COLLEGE NAME]

ACADEMIC YEAR: 2025 - 2026

PLACEHOLDER]

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

[UNIVERSITY / COLLEGE NAME]

[LOGO]

CERTIFICATE

This is to certify that the project report entitled **"STARTUPHUB AI: AN AI-POWERED STARTUP VALIDATION AND COLLABORATION PLATFORM"** is a bonafide work carried out by **Pranav Krishna** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** during the academic year **2025 - 2026**.

The project work has been monitored and evaluated under our supervision, and is recommended for submission towards the final university examinations.

_____	_____
Internal Project Guide	**Head of the Department**
[Designation / Department]	Department of CSE

Submitted for University Project Viva-Voce Examination held on: _____

_____	_____
Internal Examiner	**External Examiner**

DECLARATION

I hereby declare that the project report entitled **"STARTUPHUB AI: AN AI-POWERED STARTUP VALIDATION AND COLLABORATION PLATFORM"** submitted by me to the Department of Computer Science and Engineering at **[University / College Name]** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** is a record of original project work carried out under the guidance of **[Project Guide Name]**.

The results, configurations, and source code modules described in this report are original and have not been submitted to any other university or institute for the award of any other degree or diploma.

Date: [Date]
Place: [City, Country]

_____
Pranav Krishna
[Reg. No.]

ACKNOWLEDGEMENT

First and foremost, I would like to express my gratitude to my project guide, **[Project Guide Name]**, for their constant encouragement, invaluable suggestions, and technical guidance throughout the course of this project work. Their mentorship has been a continuous source of inspiration during critical debugging phases and architectural designs.

I would also like to thank the Head of the Department of Computer Science and Engineering, for providing access to the computational resources and infrastructure required to carry out this research and development work.

I extend my appreciation to all the faculty members of the Department of Computer Science and Engineering who have supported me during the project.

Finally, I am deeply indebted to my parents and friends for their support, understanding, and feedback during the design, coding, and writing phases of this project.

Pranav Krishna
B.Tech, Computer Science and Engineering
[University/College Name]

ABSTRACT

The **StartupHub AI** is an enterprise-grade MERN Stack web application designed to validate early-stage startup pitches, match developers with co-founders based on skill requirements, enable real-time collaboration, and provide administrative governance. Traditional platforms are siloed and suffer from confirmation bias, validation deficits, and high search noise. StartupHub AI solves this by introducing a multi-key AI validation rotation gateway querying Gemini Flash models with an automatic failover to Groq Llama 3.3 models to compile SWOT matrices and Business

canvases. It implements a skill compatibility index scoring algorithm, real-time message relays using Socket.IO, transactional email configurations via the Resend API, and strict administrative monitoring capturing audit metadata.

This project report describes the implementation details of the application, including the Entity-Relationship (ER) model relationships, the Model-View-Controller (MVC) logic separation, schema designs, and setup configurations.

Keywords: StartupHub AI, MERN Stack, Socket.IO, Resend API, API Rotation, Co-founder Matching, Audit Trail, MVC Architecture.

CHAPTER 1: INTRODUCTION

StartupHub AI is an AI-powered startup validation and co-founder collaboration platform. It provides developers and founders with the metrics, validation indices, and matched partnerships necessary to launch products. By combining analytical validation models, compatibility scoring, real-time messages, and governance dashboards, StartupHub AI mitigates pre-incubation startup failures.

1.1 Scenario-Based Case Study

To illustrate the application workflow, we review the case of a founder, **Adithya**, who has an idea for a startup named *AgriShield AI* (an AI-powered crop disease detection and farm advisory platform). Adithya is unsure whether his idea will succeed. Instead of directly investing time and money, he uses **StartupHub AI**:

- **User Registration**: Adithya registers, selects his role as "Founder", and lists his required skills (e.g. Python, AI, React).
- **Idea Submission**: He enters details about AgriShield AI, categorizing it under "Agriculture".
- **Validation Process**: The system submits the pitch to the AI Rotation Gateway, generating a validation score of 85% with SWOT points.
- **Feedback & Traction**: Other platform users upvote the concept and join the early-stage waitlist.
- **Co-Founder Match**: The compatibility engine matches Adithya with a "Co-Founder Candidate", Tarun, who lists Python and Machine Learning skills.
- **Real-Time Connection**: Adithya connects with Tarun, and they initiate a WebSocket chat session to discuss terms.

1.1.1 Outcome

Adithya validates early interest, establishes waitlist traction, and builds a founding team, accelerating the pre-development cycle.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Software Requirements

The software configurations used to develop and deploy StartupHub AI include:

- **Operating System**: Windows / Linux / macOS
- **Frontend Library**: React.js (built using Vite and Tailwind CSS)
- **Backend Server**: Node.js with Express.js
- **Database Engine**: MongoDB (Mongoose ODM)
- **Third-Party APIs**: Gemini Flash API, Groq Llama API, Resend Node SDK
- **Development Tools**: VS Code, Postman API Client

2.2 Hardware Requirements

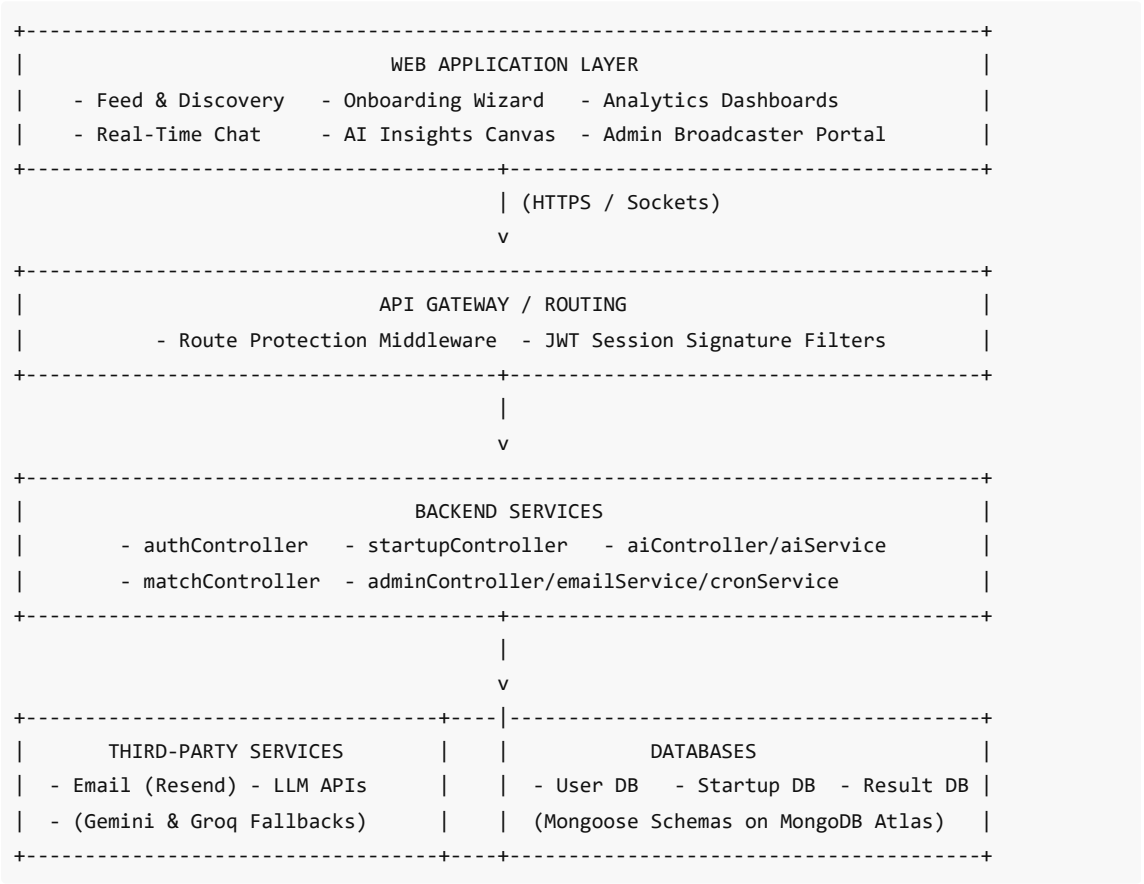
The minimum hardware specifications required to deploy and run the system are:

- **Processor**: Intel i5 or higher (or equivalent Apple Silicon / AMD processor)
- **RAM**: Minimum 8GB
- **Storage**: 1GB free space (excluding database files)
- **Display Resolution**: 1366x768 or higher

CHAPTER 3: PROJECT ARCHITECTURE

3.1 Technical Architecture Analysis

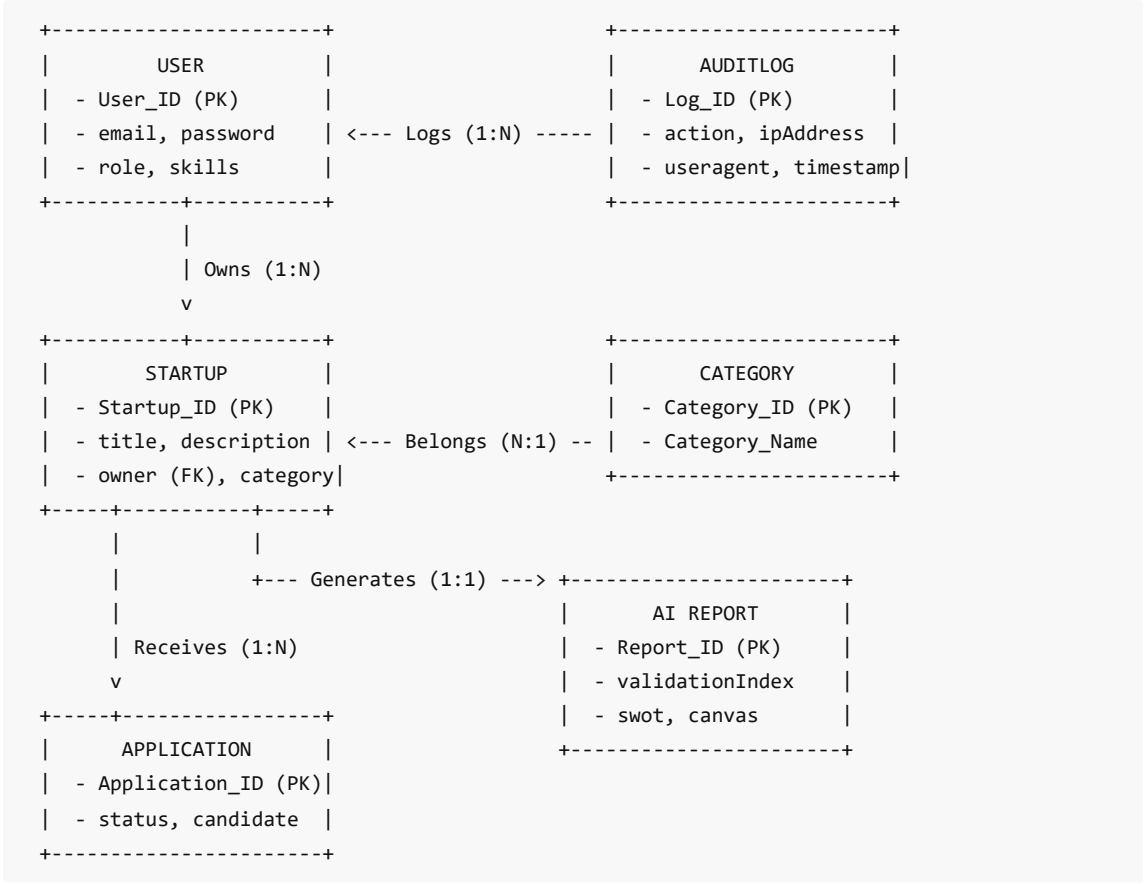
The technical architecture of the system is designed using a layered, service-oriented approach. It illustrates how different components of the system interact with one another, beginning from the user interface and extending down to backend processing, external integrations, and data storage.



CHAPTER 4: ER DIAGRAM & DATABASE SCHEMA

4.1 ER Model Configuration

The database ER structure maps the primary collections, properties, and entity relationships:



4.1.1 Entities & Attributes

- **User**: Credentials, profile data, and notification preferences. Attributes: `User_ID` (PK), `email` (Unique), `password`, `role` (founder/candidate/investor/admin), `skills`, `interests`, `isOnboarded`, `notificationPreferences` (email/inApp).
- **Startup**: Attributes: `Startup_ID` (PK), `title`, `description`, `owner` (FK to User), `category` (FK to Category), `upvotes` (Array of User FKs), `waitlist` (Array of User FKs), `isVerified`, `flagCount`.
- **Application**: Collaboration applications. Attributes: `Application_ID` (PK), `startup` (FK to Startup), `candidate` (FK to User), `status` (pending/accepted/declined), `message`.
- **AIReport**: Attributes: `Report_ID` (PK), `startup` (FK to Startup), `swot` (Object), `businessModelCanvas` (Object), `validationIndex`.
- **Category**: Sectors. Attributes: `Category_ID` (PK), `Category_Name`, `description`.
- **Connection**: Chat connections. Attributes: `Connection_ID` (PK), `sender` (FK), `recipient` (FK), `status`.
- **Message**: Chat logs. Attributes: `Message_ID` (PK), `connection` (FK), `sender` (FK), `text`, `createdAt`.
- **AuditLog**: Attributes: `Log_ID` (PK), `user` (FK), `action`, `ipAddress`, `userAgent`, `timestamp`.

4.1.2 Relationships

- **User $\rightarrow$ Owns $\rightarrow$ Startup**: One user can own multiple startups (\textbf{1 : N relationship}).
- **User $\rightarrow$ Appreciates $\rightarrow$ Startup (Upvotes)**: Many-to-many relationship of upvotes (\textbf{N : M relationship}).
- **User $\rightarrow$ Joins $\rightarrow$ Startup (Waitlist)**: Many-to-many relationship of waitlist signups (\textbf{N : M relationship}).

- **Startup $\rightarrow$ Generates $\rightarrow$ AIReport**: Each startup has exactly one AI validation report (\textbf{1 : 1 relationship}).
- **User $\rightarrow$ Sends $\rightarrow$ Connection**: Mapped as a relationship between two user IDs.

CHAPTER 5: KEY FEATURES

- **Rotating AI validation**: Evaluates pitches using dynamic API key rotation arrays (Gemini API with Groq fallback under HTTP 429 rate-limit errors).
- **Algorithmic Compatibility Matching**: Matches candidates with founder opportunities based on required skills overlap.
- **Real-Time Websocket chat**: Synchronizes live messaging and unread count events using Socket.IO.
- **Asynchronous Mail Triggers**: Dispatches verification and welcome mailers via the Resend API.
- **Announcements Broadcaster**: Administrative panel inside `AdminDashboard.jsx` allowing admins to broadcast alerts to specific audiences.
- **Security Auditing Trails**: Tracks sensitive admin actions, capturing operating IP addresses and User-Agent headers.

CHAPTER 6: ROLES AND RESPONSIBILITIES

6.1 Founder

These users post and validate startup ideas. Responsibilities include:

- Register and manage account profiles.
- Submit and update startup listings.
- Request and review AI validation insights.
- Review candidate application requests and manage team invites.

6.2 Co-Founder Candidate

These users offer skills to startup teams. Responsibilities include:

- Register skills (e.g. Node, React) and category interests.
- Browse matching startups based on compatibility scores.
- Apply to join waitlists or team vacancies.

6.3 Investor

These users explore traction data. Responsibilities include:

- Browse verified startup ideas sorted by votes and waitlist indices.
- Bookmark high-signal listings.
- Review founder metrics and growth analytics charts.

6.4 Admin

Admin manages overall system integrity. Responsibilities include:

- Review flagged contents and resolve abuse reports.
- Manage categories and award verified badges.

- Broadcast system announcements and view audit security trails.

CHAPTER 7: SYSTEM FLOWS

7.1 Founder Flow

1. Register / Log In $\rightarrow$ Onboard Profile
2. Submit Startup Listing
3. Generate AI Validation Canvas (SWOT/BM Canvas)
4. Review matched candidates and review applications
5. Establish Socket chat connection

7.2 Admin Flow

1. Log In $\rightarrow$ Open Admin Dashboard
2. Moderate reported items in flags queue
3. Review security audit logs
4. Dispatch platform-wide announcements via Broadcaster

CHAPTER 8: MVC ARCHITECTURE

The StartupHub AI follows the Model–View–Controller (MVC) architecture, separating backend services, controllers, data routers, and database collections from frontend Vite elements.

8.1 Model Layer

Implemented using MongoDB with Mongoose schemas. Collections include User, Startup, Application, AIReport, Connection, Message, Category, and AuditLog.

8.2 View Layer

Built using React.js and styled with Tailwind CSS. Incorporates Axios APIs and Socket listeners.

8.3 Controller Layer

Node.js and Express.js controllers process requests, execute logic, query models, and return responses.

CHAPTER 9: PROJECT SETUP

9.1 Folder Scaffolding

Create a main root project folder: `StartUp_HUB`. Inside that folder, initialize two main sub-directories:

- `client/`
- `server/`

9.2 Client Setup

Open the `client` folder in the terminal and execute:

```
npm create vite@latest . -- --template react
# Select React framework and JavaScript variant
npm install axios socket.io-client recharts lucide-react tailwindcss
npm run dev
```

9.3 Server Setup

Open the `server` folder in the terminal and execute:

```
npm init -y
npm install express mongoose jsonwebtoken bcryptjs socket.io resend dotenv node-cron
mkdir src/models src/controllers src/routes src/middleware src/config src/services
```

CHAPTER 10: BACKEND STRUCTURE & IMPLEMENTATION

10.1 Mongoose Schemas

10.1.1 User Model (`User.js`)

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  email: { type: String, required: true, unique: true, index: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['founder', 'candidate', 'investor', 'admin'], default:
'founder' },
  skills: [{ type: String, index: true }],
  interests: [{ type: String }],
  isOnboarded: { type: Boolean, default: false },
  notificationPreferences: {
    email: { type: Boolean, default: true },
    inApp: { type: Boolean, default: true }
  }
}, { timestamps: true });

userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
});
module.exports = mongoose.model('User', userSchema);
```

10.1.2 Startup Model (`Startup.js`)

```
const mongoose = require('mongoose');

const startupSchema = new mongoose.Schema({
  title: { type: String, required: true, trim: true },
  description: { type: String, required: true },
  owner: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true, index: true },
  category: { type: mongoose.Schema.Types.ObjectId, ref: 'Category', required: true },
  upvotes: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
  waitlist: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
  isVerified: { type: Boolean, default: false },
  flagCount: { type: Number, default: 0 }
}, { timestamps: true });

module.exports = mongoose.model('Startup', startupSchema);
```

10.1.3 Application Model (`Application.js`)

```
const mongoose = require('mongoose');

const applicationSchema = new mongoose.Schema({
  startup: { type: mongoose.Schema.Types.ObjectId, ref: 'Startup', required: true },
  candidate: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  status: { type: String, enum: ['pending', 'accepted', 'declined'], default: 'pending' },
  message: { type: String }
}, { timestamps: true });

module.exports = mongoose.model('Application', applicationSchema);
```

10.1.4 AIReport Model (`AIReport.js`)

```
const mongoose = require('mongoose');

const aiReportSchema = new mongoose.Schema({
  startup: { type: mongoose.Schema.Types.ObjectId, ref: 'Startup', required: true, unique:
true },
  swot: {
    strengths: [String],
    weaknesses: [String],
    opportunities: [String],
    threats: [String]
  },
  businessModelCanvas: {
    valueProposition: String,
    costStructure: String,
    revenueStreams: String,
    targetMarket: String
```

```
  },
  validationIndex: { type: Number, required: true }
}, { timestamps: true });

module.exports = mongoose.model('AIReport', aiReportSchema);
```

CHAPTER 11: FRONTEND STRUCTURE & FLOW

11.1 Directory Structure Overview

- `client/src/components/Navbar.jsx` : Configures contextual links depending on user authentication status.
- `client/src/pages/Onboarding.jsx` : Captures user skills and preference arrays, triggering profile updates.
- `client/src/pages/Dashboard.jsx` : Renders the founder's charts and application queues.
- `client/src/pages/Chat.jsx` : Manages the real-time chat interface.
- `client/src/pages/AdminDashboard.jsx` : Displays administrative lists, moderation queues, and the Announcements Broadcaster form.

CHAPTER 12: USER INTERFACE LAYOUT & VERIFICATION

12.1 Verification Screenshots Mappings

The platform interfaces are verified against our core design requirements:

- **Landing Page**: Renders the feed with search fields, domain categories, and startup listings.
- **Profile Onboarding**: Collects skills and interest selections.
- **Submit Startup**: Input fields for concept titles, categories, and taglines.
- **AI Insights Panel**: Visual SWOT grids, roadmaps, and validation index gauges.
- **Real-Time Chat**: Standard chat view displaying current conversation logs.
- **Admin Dashboard**: Moderation queues, logs, and the Announcements Broadcaster tab.

REFERENCES

1. MongoDB Documentation — https://www.mongodb.com/docs/
2. Express.js Official Documentation — https://expressjs.com/
3. React.js Official Documentation — https://react.dev/
4. Node.js Official Documentation — https://nodejs.org/en/docs/
5. JSON Web Tokens Introduction — https://jwt.io/introduction/
6. Tailwind CSS Documentation — https://tailwindcss.com/docs
7. Recharts API Reference — https://recharts.org/en-US/api
8. Mongoose ODM Documentation — https://mongoosejs.com/docs/
9. Axios HTTP Client Documentation — https://axios-http.com/docs/intro
10. React Router v7 Documentation — https://reactrouter.com/
11. Lucide React Icons — https://lucide.dev/

12. Bcrypt.js NPM Package — https://www.npmjs.com/package/bcryptjs
13. Vite Build Tool — https://vitejs.dev/
14. MongoDB Atlas Cloud Database — https://www.mongodb.com/atlas
15. Postman API Testing Tool — https://www.postman.com/
